

Отчет по технической реализации приложения Climo

1. Общая характеристика проекта

`Climo` - мобильное Flutter-приложение для пользователей, чувствительных к изменениям погоды. Основная задача приложения - показывать индекс благополучия, объяснять погодные и геомагнитные факторы риска, давать спокойные рекомендации и позволять пользователю вести дневник самочувствия.

Текущая версия реализована как MVP-прототип. В коде уже есть основная логика интерфейса, расчет риска, получение погодных данных, экран рекомендаций, дневник, профиль пользователя и базовые тесты. Проект ориентирован на iOS/Android, но также может запуститься на других Flutter-платформах.

Главный файл реализации: `lib/main.dart`. Тесты находятся в `test/widget\_test.dart`.

2. Используемые технологии

Приложение написано на Flutter и Dart. Для интерфейса используются стандартные компоненты `Material 3` и `CupertinoIcons`, что позволяет быстро получить мобильный интерфейс, похожий на iOS-приложение, без подключения тяжелых UI-библиотек.

В `pubspec.yaml` подключены:

- `flutter` - основной SDK для мобильной разработки;
- `cupertino\_icons` - набор iOS-подобных иконок для нижней навигации;
- `http` - выполнение сетевых запросов к погодным и геомагнитным API;
- `flutter\_test` и `flutter\_lints` - тестирование и статическая проверка кода.

Дополнительные пакеты для графиков, роутинга или state management пока не добавлялись. Это сделано намеренно: текущий MVP небольшой, поэтому состояние хранится внутри `StatefulWidget`, а график реализован через `CustomPainter`.

3. Архитектура приложения

Текущая версия собрана в одном файле `lib/main.dart`, но внутри кода уже выделены логические слои:

- модели данных;
- сервис получения погоды;
- движок расчета риска;
- движок подбора рекомендаций;
- экраны приложения;
- переиспользуемые UI-компоненты;
- демо-данные для fallback-режима.

Такой подход удобен для MVP и демонстрации. При дальнейшем развитии проекта код рекомендуется разнести по feature-first структуре:

```
lib/  
  app/  
  core/  
  features/  
    dashboard/  
    forecast/  
    diary/  
    recommendations/  
    profile/  
    risk_engine/  
  shared/
```

Основная точка входа - `ClimoApp`. Он создает `MaterialApp`, подключает тему `ClimoTheme` и открывает `ClimoShell`. `ClimoShell` управляет текущей вкладкой нижней навигации, хранит профиль, прогноз, дневник и запускает загрузку реальных погодных данных.

4. Модели данных

В приложении используются следующие основные модели:

- `UserProfile` - профиль пользователя: имя, возраст, город, чувствительность к гипотонии, гипертонии, головным болям, суставной чувствительности и настройки уведомлений;
- `WeatherRecord` - погодная запись: температура, перепад температуры, давление, перепад давления, влажность, ветер, Кр-индекс и источник данных;
- `RiskFactor` - отдельный фактор риска, например давление или геомагнитная активность;
- `RiskResult` - итоговый индекс риска: балл, уровень, причины, список факторов и текстовое объяснение;
- `DiaryEntry` - запись дневника самочувствия;
- `RecommendationItem` - рекомендация с категорией, тегами, профилями пользователя и приоритетом.

Эти модели позволяют отделить предметную логику от виджетов. Например, экран только отображает `RiskResult`, а сам расчет выполняется отдельно в `RiskEngine`.

5. Получение погодных и геомагнитных данных

В текущей реализации отдельный backend не используется. Приложение получает данные напрямую с открытых API:

- Open-Meteo Geocoding API - поиск координат города;
- Open-Meteo Forecast API - прогноз погоды;
- NOAA SWPC - данные Кр-индекса.

За это отвечает класс `WeatherApiService`.

Сначала метод `fetchForecast(city)` получает координаты города через Open-Meteo Geocoding. Затем загружает актуальный Кр-индекс из NOAA SWPC. После этого формирует запрос к Open-Meteo Forecast и преобразует ответ API в список `WeatherRecord`.

Важная особенность: в приложении нет API-ключей. Это снижает риск утечки секретов и подходит для MVP, потому что выбранные источники позволяют получить базовые данные без приватного ключа.

Также реализован fallback: если сеть недоступна или API вернул ошибку, приложение оставляет демо-прогноз из `sampleForecast()` и показывает пользователю карточку состояния через `WeatherStatusCard`. Это важно для мобильного приложения, так как пользователь не должен видеть пустой экран при проблемах с сетью.

6. Risk Engine

Расчет риска реализован в классе `RiskEngine`. Алгоритм детерминированный, без машинного обучения. Он подходит для первой версии, потому что понятен, объясним и легко тестируется.

Risk Engine учитывает четыре ключевых фактора:

- перепад атмосферного давления;
- геомагнитную активность по Kp-индексу;
- резкий перепад температуры;
- высокую или низкую влажность.

Каждый фактор получает уровень `RiskLevel`:

- `low` - низкий риск;
- `medium` - умеренный риск;
- `high` - высокий риск.

Пороги частично зависят от профиля пользователя. Например, для пользователей с гипотонией или гипертонией порог чувствительности к перепаду давления ниже. Для пользователей с суставной чувствительностью ниже порог влажности.

Итоговый риск рассчитывается так:

- если активных факторов нет, риск низкий;
- если активен один фактор, риск умеренный;
- если есть красный фактор или одновременно активны несколько факторов, риск высокий.

После этого формируется балл индекса благополучия по шкале 0-10. В текущем MVP низкий риск отображается как `3/10`, умеренный как `4/10`, высокий как значение от `6/10` и выше в зависимости от количества активных факторов.

Кроме балла, `RiskEngine` возвращает объяснение: какие факторы повлияли на риск и почему пользователь видит предупреждение. Это соответствует основной задаче приложения - не просто показывать цвет, а объяснять причину.

7. Рекомендации

Подбор рекомендаций реализован в классе `RecommendationEngine`.

В приложении есть список `recommendations`. Каждая рекомендация имеет:

- категорию;
- текст;
- погодные теги;
- профильные теги;

- приоритет.

Категории рекомендаций:

- физическая активность;
- питание и гидратация;
- бытовые советы и режим;
- контроль самочувствия.

Алгоритм выбирает рекомендации по тегам активных факторов риска. Например, если сработал фактор ``pressure_drop``, пользователю показываются советы, связанные с перепадами давления. Если рекомендация предназначена только для гипотоников или гипертоников, она дополнительно фильтруется по профилю.

В рекомендациях соблюдено ограничение по медицинским формулировкам: текст про лекарства содержит фразу "по назначению врача". На экране рекомендаций также есть предупреждение, что советы носят информационный характер и не заменяют консультацию врача.

8. Frontend и UX

Интерфейс реализован в спокойной минималистичной стилистике по предоставленным скриншотам. За общую тему отвечает ``ClimoTheme``.

Основные визуальные решения:

- светло-голубой фон;
- крупная типографика;
- белые карточки с мягкими скруглениями;
- нижняя навигация из пяти вкладок;
- цветовая индикация риска: зеленый, желтый, красный;
- мягкие пастельные карточки дневника и рекомендаций;
- минимум анимаций.

Главная навигация реализована через ``NavigationBar`` с вкладками:

- Главная;
- Прогноз;
- Советы;
- Дневник;
- Профиль.

Для сохранения состояния вкладок используется ``IndexedStack``. Благодаря этому при переключении между разделами экран не пересоздается полностью.

8.1. Главный экран

Экран ``DashboardScreen`` показывает:

- приветствие и дату;
- источник данных;

- состояние загрузки погоды;
- карточку индекса благополучия;
- список факторов влияния;
- короткое объяснение текущего риска.

Главная карточка `RiskHero` окрашивается по уровню риска. В ней отображается индекс благополучия, например `4/10`, и спокойное предупреждение: "Обратите внимание на самочувствие".

8.2. Экран прогноза

Экран `ForecastScreen` показывает график индекса благополучия и прогноз на 5 дней. График реализован вручную через `TodayRiskChartPainter`, без сторонней библиотеки. Это уменьшает зависимости и достаточно для MVP.

Для каждого дня отображается температура и индекс риска. Цвет индекса зависит от уровня риска.

8.3. Экран рекомендаций

Экран `RecommendationsScreen` показывает предупреждение о том, что рекомендации не заменяют врача, а затем группирует советы по категориям. Каждая группа оформлена отдельной карточкой с иконкой и цветовым акцентом.

8.4. Дневник самочувствия

Экран `DiaryScreen` позволяет создать запись:

- выбрать самочувствие по шкале 1-10;
- отметить симптомы через `FilterChip`;
- добавить текстовый комментарий;
- сохранить запись.

История записей отображается карточками `DiaryEntryCard`. Цвет карточки зависит от самочувствия:

- зеленый - хорошее;
- желтый - среднее;
- красный - плохое.

В демо-данных уже есть несколько записей за 2-4 мая 2026 года, чтобы показать внешний вид дневника.

8.5. Профиль

Экран `ProfileScreen` хранит настройки чувствительности пользователя:

- гипотония;
- гипертония;
- головные боли;
- уведомления.

Эти параметры влияют на расчеты Risk Engine и подбор рекомендаций. Например, включенная гипотония снижает порог реакции на перепады давления.

9. Уведомления

В текущей версии реализована только UI-часть настроек уведомлений в ``UserProfile`` и ``ProfileScreen``. Полноценные push-уведомления пока не подключены.

В дальнейшем можно добавить:

- ``flutter_local_notifications`` для локальных уведомлений;
- Firebase Cloud Messaging для серверных push;
- расписание предупреждений по времени;
- защиту от повторяющихся одинаковых уведомлений.

Сейчас профиль уже содержит флаги ``notificationsEnabled``, ``notifyPressure``, ``notifyKp``, ``notifyHumidity``, ``notifyTemperature``, поэтому модель данных готова к расширению.

10. Статистический минимум и персонализация

В коде уже есть база для будущей персонализации: ``DiaryEntry`` связывает самочувствие пользователя с погодой и рассчитанным риском. Это позволит позже анализировать совпадения между плохим самочувствием и погодными факторами.

На первом этапе можно реализовать простой статистический алгоритм без нейросети:

- считать, сколько раз пользователь отмечал плохое самочувствие при перепаде давления;
- считать совпадения плохого самочувствия с высоким Кр-индексом;
- выводить в профиле простые наблюдения, например: "Плохое самочувствие чаще совпадало с перепадами давления".

Такой подход лучше подходит для MVP, чем сложная ML-модель: он прозрачен, не требует большого объема данных и легко объясняется пользователю.

11. Обработка ошибок и устойчивость

Сетевые запросы обернуты в обработку ошибок. Если API недоступен, ``ClimoShell`` сохраняет демо-данные и записывает сообщение в ``_weatherError``. На главном экране отображается ``WeatherStatusCard`` с объяснением и кнопкой повторной загрузки.

Это важно для пользовательского опыта: приложение остается полезным даже без сети, а пользователь понимает, почему данные могут быть демонстрационными.

12. Тестирование

В проекте есть тесты в ``test/widget_test.dart``.

Покрыты два базовых сценария:

- widget-тест проверяет запуск приложения, отображение главного экрана и переходы между вкладками;
- unit-тест проверяет, что Risk Engine выставляет высокий риск при сочетании сильного перепада давления, температуры, влажности и высокого Кр-индекса.

Для проверки качества используются команды:

```
dart format .  
flutter analyze  
flutter test
```

Текущая реализация была проверена этими командами.

13. Ограничения текущей версии

Текущая версия является MVP и имеет ряд ограничений:

- нет постоянного локального хранения дневника;
- нет полноценного onboarding;
- профиль хранится только в состоянии приложения;
- push-уведомления пока не подключены;
- график на экране прогноза пока статический по форме;
- нет backend для централизованного обновления прогноза;
- медицинские источники не вынесены в отдельный файл документации;
- код пока находится в одном файле, хотя логически уже разделен на блоки.

Эти ограничения не мешают демонстрации основной идеи, но должны быть устранены перед полноценным релизом.

14. Возможное развитие

Следующие шаги технической реализации:

1. Разнести код по feature-first архитектуре.
2. Добавить локальное хранилище для профиля и дневника.
3. Подключить локальные уведомления.
4. Сделать экран onboarding с анкетой пользователя.
5. Сделать график прогноза динамическим на основе реальных данных.
6. Добавить простой анализ персональных триггеров по дневнику.
7. Вынести медицинские источники в `docs/sources.md`.
8. Расширить тесты для Risk Engine, рекомендаций и дневника.

15. Вывод

В текущем виде приложение `Climo` уже реализует основу продукта: получает погодные и геомагнитные данные, рассчитывает индекс риска, показывает понятный интерфейс, объясняет факторы влияния, предлагает рекомендации и позволяет вести дневник самочувствия.

Техническая реализация соответствует идее спокойного и доступного приложения для метеочувствительных пользователей. Основной упор сделан на простую архитектуру, объяснимый алгоритм риска, понятные визуальные состояния и готовность к дальнейшему расширению.